```
        },
    ))
    log.Fatal(http.ListenAndServe(":4242", nil))
}
```

At this point, you know how to instrument the `echo` service to emit metrics, but how do these metrics arrive at a destination where you can consume them? Let's take a look in the next section.

2.4.2 Telemetry

We are now in the position to execute the `echo` service locally and expose the Prometheus metrics:

```
$ ./echosvc
```

We can scrape the metrics of our `echo` service using Prometheus (http://prometheus.io/), which, in turn, can act as a data source in Grafana as the destination. To set up the destination, use the Docker Compose manifest file shown in the next listing (which references listing 2.9) to set up Prometheus and Grafana.

Listing 2.8 Using docker-compose.yaml to stand up Prometheus and Grafana

```
version: "3.5"
networks:
  prom:
services:
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prom-config.yaml:/etc/prometheus/prometheus.yml
    networks:
      - prom
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    networks:
      - prom
```

Like we've seen for logs, we also need to configure the telemetry agent for metrics. In this case, we this do by defining the Prometheus scrape config, as shown in the prom-config.yaml file in the following listing.

Listing 2.9 Prometheus configuration file prom-config.yaml

```
global:
  scrape_interval: 5s
  evaluation_interval: 5s
```